

Cellular Automata for Multiple Biome Cave Generation in Video Games

Sanz, Reagan
CS581 Bioinspired Computation
University of Tennessee
Knoxville, TN, USA
rsanz@vols.utk.edu

Abstract— Open world and infinite map video games often depend on procedural generation of their terrain to create interactive and dynamic gameplay for its players. The generation of these worlds needs to be complex and interesting, while also computationally efficient to prevent slow response time when new sections of the maps are generated. Many games often implement underground “cave systems” for players to explore. Cellular Automata (CA), which are inspired by the self-reproducing behavior of living organisms in nature, can be used in the generation of these cave systems in video games. CAs are computationally efficient and their transitions between states can create complex and elaborate cave generation. This paper focuses on how different CA rules and parameters can be implemented for dynamic cave generation across different “biomes” within the game. Thus, allowing for uniquely generated caves depending on the player’s location. Four biomes are introduced, each with specific themes that correlate with their specific cave system parameters.

Keywords— *Cellular Automata, Video Games, Procedural Generation, Terrain Generation, Caves in Video Games*

I. INTRODUCTION / MOTIVATION

Video games have become a prevalent aspect of modern culture across all age ranges. This industry has been continuing to grow and according to the Entertainment Software Association (ESA), the global market for video games totaled \$159 billion USD in 2020 alone [5]. This large industry continues to expand, and a main challenge amongst game developers is ensuring the longevity of their games. Open-world is a popular genre of video games that allow users more freedom in exploration and creativity in user world-building. However, designing these generated worlds completely by hand is not feasible for developers and artists, especially within this demanding, fast-paced industry. For this reason, many open-world games implement procedural content generation (PCD). Freiknect defines PCG as “[the] automatic creation of digital assets for games, simulations or movies based on predefined algorithms and patterns that require a minimal user input” [6]. These procedural techniques can aid in the creation of complex games through algorithms instead of human designers. This reduces the burden on game content developers. It allows games to be released within strict deadlines but still allows designers creative control over the generated worlds through parameter changes. [7]

Procedurally generated video games are often dependent on complex, pseudo-randomized world generation to keep the game engaging to players. Many open-world games, such as Minecraft or Terraria, implement cave systems that greatly expand the world and gameplay for users. These cave systems are often generated using Perlin Noise [1], however, this paper shows how Cellular Automata (CA) can also be implemented in cave generation. The interest of this topic stems from the importance of procedural world generation of video games in order to increase player enjoyment and extend the games’ longevity.

While CAs have been introduced in video game generation before, this paper focuses on the impact of different rules and parameters on the formation of caves. Four different biomes are introduced, each with a unique theme and specifically generated caves that match the atmosphere of the selected biome.

II. RESEARCH QUESTIONS

The research questions this paper addresses are as follows:

RQ1: Can Cellular Automata be used in the generation of pseudo-random cave systems in video games?

RQ2: How can different CA rules and parameters be implemented to capture the theme of individual “biomes” and thus generate unique caves for each one?

III. RELATED WORKS / BACKGROUND

A. Cellular Automata

Cellular Automata was originally introduced by John von Neumann in the late 1940s as models of self-reproducing organisms. Cellular automata take their inspiration from nature, such as the process of self-replication of nucleic acid or the spread of virus/bacteriophage particle [3].

To begin with, CAs were introduced in as a one-dimensional array where each state (0 or 1) was dependent on its neighboring state(s) to determine its transition. Von Neumann introduced the 2-d Cellular Automata which operates on a 2d matrix where each state is dependent on the neighbors in its surrounding area. The Von Neumann Neighborhood specifically outlines the four adjacent cells (up, down, right, and left) of the current cell, and

these are used alongside rules to determine the transition state of the cell [2].

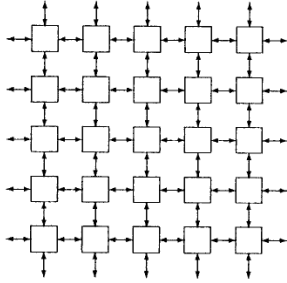


Figure 1: A 2D CA with von Neumann (orthogonal) neighborhood [2]

Cellular Automata have the capability to simulate Turing machines (perform calculations), track the path of viruses or forest fires, and model figures within biological and chemical research [2][11].

B. Video Game Cave Generation

Maps are vital in the creation of video game world-building, and for dungeon crawler and open-world games, sprawling caves are used within procedural world generation. For many games, their re-playability and enjoyment to players is based largely in their ability to generate new terrain.

For many video games, procedural terrain generation is done through 2D procedural noise functions (such as the most commonly used Perlin noise). This method “generates noise by sampling and interpolating points in a grid of random vectors” [1]. This method allows for smooth and natural-looking generation of mountains, textures, and environments in video games.

Other methods in generating terrain, and specifically caves in general, are Fractals (an example being mid-point displacement algorithms). These are mostly chosen for real-time world generation for their computational efficiency that allow for quickly generating terrain in an ever-changing environment. Despite its efficiency, it can only be controlled by a single parameter, randomized “seed”. This limits the fine-tuned control developers have over its generation [4]. Another method is Search-based procedural content generation (SBPCG). While this solves the issue that fractals face by allowing for high levels of controllability, it suffers from very high computation time and cost to generate terrain. Due to this, it’s not feasible for use in real-time generation [4].

The limitations of these previous methods have led to the introduction of Cellular Automata for infinite cave generation in these video game games. Johnson et al, introduces this method of procedural content generation (PCG) that utilizes cellular automata to create the floors and rocks of the cave systems in real-time [4].

The approach introduced by Johnson, et al. initializes floor cells into the 50x50 grid and looks at the 8 neighbors of each cell in determining if the next cell should be a rock or wall. Caves

can be customized by changing various variables of initial percent of rock cells, CA iterations, neighborhood thresholds and Moore neighborhood size. Wall cells are then defined after the cellular automata algorithm to line along the edges of floor cells (has one neighboring floor cell). This method successfully generates different caves at a fast rate (the 50x50 caves can be generated in 4.1×10^{-1} milliseconds). This is comparable to random generations that are formed in 1.4×10^4 milliseconds. The efficiency of this implementation allows for effective real-time generation of caves in video games, such as the example in this paper of *Cave Crawler*, which takes inspiration from other game mechanics (like *Diablo* and *Castle Crashers*) [4].

IV. METHODS

This paper builds on the previous work done by Johnson, et al, but with a focus on how different cellular automata rules and transitions can result in a variety of generated cave systems. The overall goal is to capture the intended “atmosphere” and mood of each biome with the uniquely generated cave systems.

A. Four “Biome” Creations

It was decided that 4 unique “biomes” would be created: Forest, Desert, Underwater, and Mountain. This project was inspired by many open-world procedural generated games (such as Minecraft) that implement unique terrain generation depending on the “biome” the player is currently located in. For this experiment, each biome had a unique “mood” to it that we wanted to capture with Cellular Automata parameters. The overall atmosphere we hoped to simulate in each biome is as follows:

1. **Forest:** Sprawling, “vine-like” caves. A decent chance of water, enemies, and treasure. A good starting cave for players. Luscious green and grassy atmosphere.
2. **Desert:** Very small patches of sporadic caves within sandy walls. Very little water or enemies. A large amount of treasure to offset the smaller caves.
3. **Underwater:** Sprawling, water-filled caves. A decent chance enemies, but very little chance of treasure. Maintains a focus on water filled caverns.
4. **Mountain:** Large, open caverns. A more difficult environment with increased enemies and low chances of treasure. Grey, rocky walls.

B. Cellular Automata Code

For generating the caves, C++ code was written to implement a CA through the Von Neumann neighborhood method. To begin with, each cell in the 64x100 matrix is randomly assigned ALIVE or DEAD according to a set chance (ranging across the different biomes)- where an ALIVE state means the wall of the cave, and a DEAD state means the open cave itself. Each cell then looks at the values of its 8 surrounding neighbors (and their states) in determining what state the current cell should transition to. An image of a cell and its neighbor states can be seen below. This method of Cellular Automata for

cave generation was based on the work by Johnson, et al [4] and Avery's video explanation of 2D procedural cave generation [10].

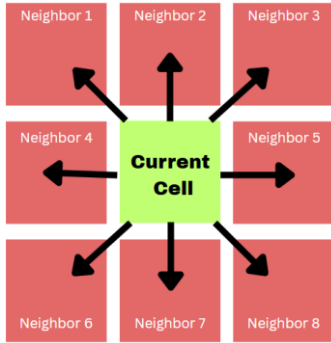


Figure 2: Neighbor Cells used in State Transitions

If a current cell's neighbors exceed a certain number of DEAD or ALIVE, it results in the cell transitioning to DEAD or ALIVE. These transitions are also determined by the corresponding biome, and more information on the specific parameter used will be described later. Once this matrix is filled of the initial randomized ALIVE or DEAD values, the CA process is repeated for a total of $it = 5$ iterations to smoothen out the cave system. A variety of different it values were tested, before 5 was landed on as computationally fast and accurate result.

Originally, the walls and cave systems are stored as ASCII characters representing the "pixel" or "block" at that point. Eventually, additional features were generated procedurally as well (including enemies, water, and treasure). Each of these "blocks" in the game were represented by unique ASCII characters. In our simulation, these characters were then converted into colored pixels and saved as a PNG through `stb_image_write.h` from the open-source GitHub code by nothings/stb [8]. This header allowed for a more visually appealing simulation of the cave generation by converting ASCII characters to colored pixels, and saving the image as a png.

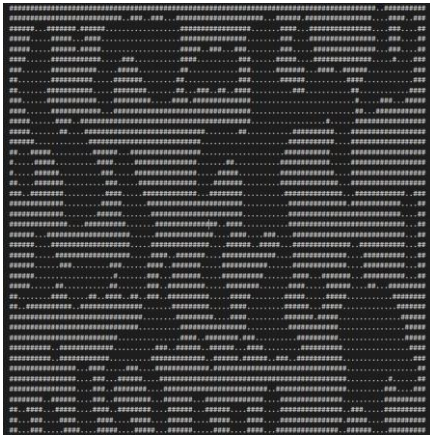


Figure 3: Original ASCII Representation of Cave

C. Additional Cave Feature Generation (Water, Enemies, and Treasure)

In addition to the cave ground and walls being generated, additional features were implemented into the cave to simulate an interactive map in a video game.

After the initial cave was generated and 5 iterations of CA were applied with different transition states, the matrix map consisted of 1's and 0's representing the walls and cave floors respectively. In order to further simulate unique biome generation, pools of "water" (blue pixels) were generated using another Cellular Automata layered on the first one. This ensured that the water cells spawned in segments only within the open caves (and not the walls). The initial chance of water spawns varied across the different biomes.

After the water cells are generated, the program then spawns in "enemies" at a set random chance across the different biomes. The program iterates through every DEAD (cave) space, and randomly spawns enemies in each tile at a rate of 1-5% (depending on the biome).

Lastly, "Treasure rooms" have a random chance of spawning in each "chunk" of the biome (in this case, a chunk represents the simulated map matrix of 64x100 blocks). While each biome had a different chance of these rooms appearing, the desert one maintained a 100% chance of treasure for every chunk generated.

D. Unique Biome Parameters

To determine the optimal CA and additional feature parameter values, mostly trial-and-error was used until the generated caves matched the intended atmosphere of each biome. Each biome was saved as a struct containing the following features

- Biome Name
- Initial Alive Probability of Walls in each cell
- Number of Alive neighbors to cause transition
- Number of Dead neighbors to cause transition
- Wall Color
- Floor Color
- Initial Alive Probability of Water in each cave cell
- Chance of Monster spawning (per block)
- Chance of Treasure spawning (per chunk of map)

For each biome, the general parameters of each were tested until they visually represented the overall theme of each biome. In addition to the parameters, different colors for wall and floors are specified per biome. This is intended to represent different types of materials (grass, sand, underwater soil, and rock respectively) of the different blocks in different environments. The corresponding color pallets are as follows

Table 1: Colors for Cave Wall/Floor Materials Across 4 Biomes

Biomes	Cave Wall and Floor Materials	
	Cave Walls	Cave Floor (Empty space)
Forest	 Grass Terrain	 Cave Space
Desert	 Sand Terrain	 Cave Space
Underwater	 Underwater Soil Terrain	 Cave (Water) Space
Mountain	 Rock Terrain	 Cave Space

Each of these blocks is implemented through the open-source nothings/stb code (see Related Works) [8], and these represent an individual block generated by a corresponding ASCII character. While we just used as flat pixels for this simulation, the intention is that these different terrain blocks would represent completely textured elements in a fully rendered game, and the enemies and characters would be fully animated assets. For sake of simplicity and the main focus of the project being the cave generation, these simplified pixels were used in place of more complex textures.

V. RESULTS

A. Optimal Biome Parameters

After trail-and-error testing multiple different Cellular Automata and feature parameters across the four different biomes, eventually the optimal parameters for each biome theme were found. The overall goal was to capture the perceived theme of each biome to create a more engaging gameplay experience for users. The results of the four biomes can be seen below in Table 2 and Table 3.

Table 2: Cellular Automata Parameters for Each Biome

Biomes	Cellular Automata Parameters	
	Initial ALIVE (wall) Chance	Transition (Neighbor) Threshold (Neighbors -> Current)
Forest	53%	3+ DEAD -> DEAD 4+ ALIVE -> ALIVE
Desert	65%	5+ DEAD -> DEAD 4+ ALIVE -> ALIVE
Underwater	50%	4+ DEAD -> DEAD 4+ ALIVE -> ALIVE
Mountain	40%	2+ DEAD -> DEAD 4+ ALIVE -> ALIVE

Table 3: Additional Feature Parameters for Each Biome

Biomes	Additional Feature Parameters	
	Enemy Spawn Rate (per block)	Cave Floor (Empty space)
Forest	1%	Forest
Desert	5%	Desert
Underwater	1%	Underwater
Mountain	1%	Mountain

These parameters were able to successfully generate caves across the four biomes, while maintaining the intended atmosphere of the individual biome. This confirms the answer to RQ2

RQ2- Using different Cellular Automata and Feature Parameters succeeds in generating unique caves across the four different biomes

B. Biome Results

Some of the results of the generated caves across all four biomes can be found below. To begin with, a key for image generation is as follows (Table 4). “Enemies” represent an adversary for the player (such as a monster or creature), water is generated through an additional CA in order to form in small pools that the player can interact with, and the treasure rooms spawn, at most, once per biome “chunk” (64x100 map region) with treasures inside to aid the player in exploration or combat.

Table 4: Key for Simulated Cave Generation

KEY	
Enemy	
Water	
Treasure Room	

Forest: The forest cave is surrounded by grassy walls and features sprawling caves. It contains an average chance of treasure and enemies. This is intended to be a fun, yet not too challenging cave for starting players.

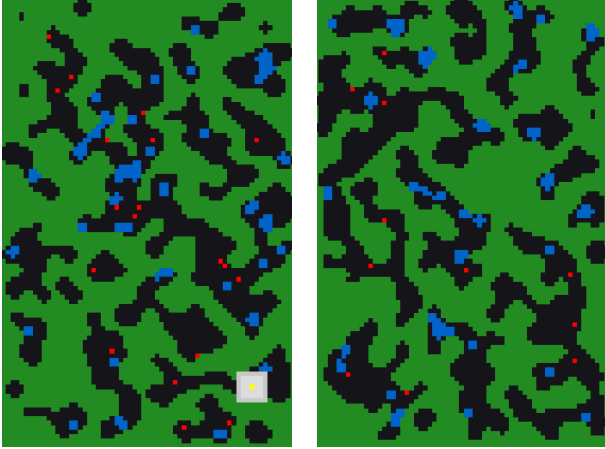


Figure 4: Example of Forest Generated Caves

Desert: The desert consists of majority sandy walls. This is indicated by Table 2 that shows desert biomes with the highest chance of “ALIVE” cells (where these cells represent the cave walls). This biome will also contain a “treasure” room 100% of the time when generating. The enemy spawn rate for the cave pixels is also slightly higher (5%) in this biome, partially due to the lower number of cave (DEAD) states total.

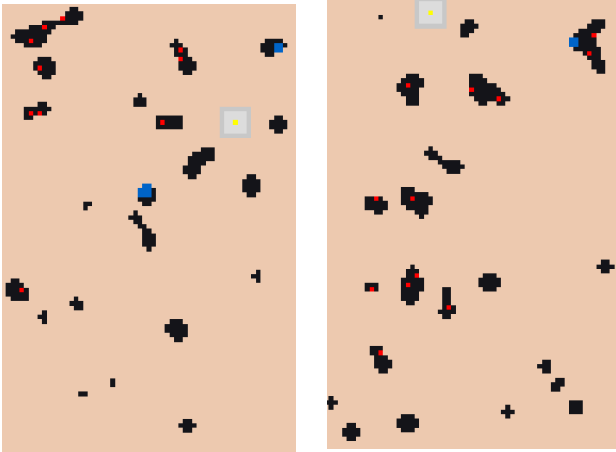


Figure 5: Example of Desert Generated Caves

Underwater: The underwater caves contain sprawling, more open, water-filled caves (100% chance of water in each empty cave block), with a low chance of treasure.

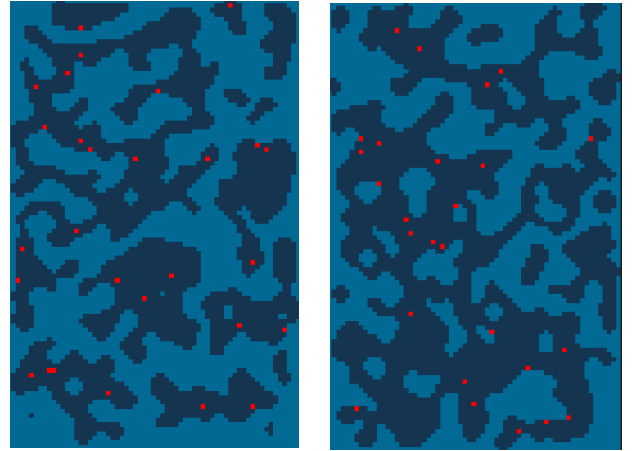


Figure 6: Example of Underwater Generated Caves

Mountain: The last biome, the mountain one, is intended to be more difficult. The spawn rate for enemies maintains the 1% chance, but due to the large open caves, this results in a higher number of enemies per chunk. This biome is intended to be populated by large caverns with many enemies- possibly representing a difficult level or increased challenge for players.

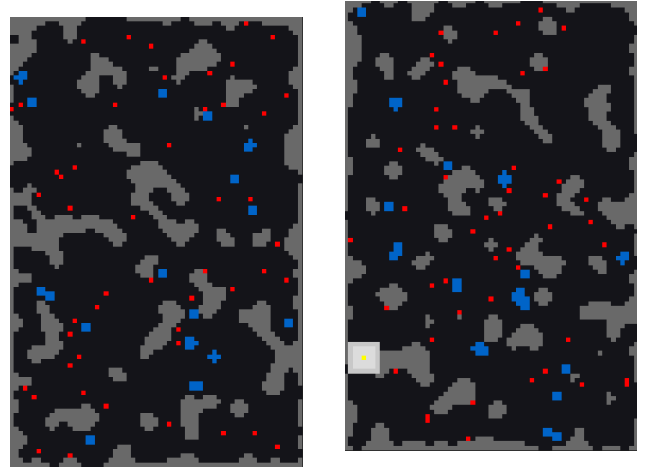


Figure 6: Example of Mountain Generated Caves

VI. FUTURE WORK

A. Additional Transition States

This implementation applies the Von Neumann neighborhood method by viewing the 8 surrounding neighbors of the current cell and then determining its transition state based on the states of these neighbors to the north, south, east, west, and corners of the cell. While this worked for these caves, additional rules or criteria could have been implemented to create even more unique and diverse caves across the different biomes. In future work, these methods of looking at more or different neighboring cells, or adding complexity to the transition rules, could allow for even more dynamic and interesting cave generation.

B. Video Game Simulation

An original goal of this paper was to implement these generated caves into a simulated video game using either Unity or Godot. Due to time constraints, this was not implemented, and images of the caves were used instead. For a possible future work, these caves could be generated in real time through a “Dig Dug” styled game (Where the map is viewed from a side view, with players having the ability to move in the x-y direction, and enemies within the cave walls). An image of a Dig Dug style arcade game can be seen below.



Figure 7: Screenshot of "Dig Dug" by Namco [9]

C. Additional Features

Originally, it was planned that lava and ores would also spawn into the caves at specific rates across the different biomes. Specifically, these features would take into account not only if a state is ALIVE or DEAD (cave wall or cave floor), but also take in consideration the depth of the elevation itself. The lower rows (90-100) would have higher rates of ores or lava spawning, and a mathematical formula would be used to calculate spawn rates depending on the depth. This feature was not implemented both due to time constraints and due to the limitations of the color “pixels” (IE: the red of the enemies vs. the lava). In a larger, more generated image or game, the dimensions of the generated caves would allow for more complex features without feeling visually overwhelming.

VII. CONCLUSION

Many procedural, open-world video games depend on complex and interesting world generation in order to extend the longevity and entertainment of the game for its players. Terrain generation has historically been done through both manual design and algorithmic generation. Procedural generation tends

to be preferable to developers to create complex worlds at a faster pace to meet deadline goals, and the use of cellular automata can be used to build the sprawling caves that many open-world games implement into their world-building. These allow for complex cave systems that can be generated efficiently in real-time without causing additional lag or interrupting gameplay. This paper shows how different feature and cellular automata parameters can be implemented to generate even more unique cave structures across a variety of “biomes” within the world, thus capturing the atmosphere of each biome and creating a more enriching world for players to explore.

REFERENCES

- [1] F. Güler, E. Onbaşıoğlu, “Applying Perlin Noise on 3D Hexagonal Tiled Maps,” IEEE, International Symposium on Multidisciplinary Studies and Innovative Technologies (ISMSIT), 2022, Ankara, Turkey, <https://ieeexplore.ieee.org/document/9932712/authors#authors> [Accessed 5/6/2026]
- [2] P. Sarkar, “A Brief History of Cellular Automata”, ACM, ACM: Computing Surveys, Vol 32, Issue 1, 2000, Indian Statistical Institute, <https://dl.acm.org/doi/epdf/10.1145/349194.349202> [Accessed 5/6/2026]
- [3] D. Marinescu, “Complex Systems and Clouds: Ch 2- Nature Inspired Algorithms and Systems”, ScienceDirect, 2017, Pages 33-63, <https://www.sciencedirect.com/science/chapter/monograph/abs/pii/B9780128040416000025> [Accessed 5/6/2026]
- [4] L. Johnson, G. Yannakakis, and J. Togelius. “Cellular automata for real-time generation of infinite cave levels”, Unknown Publisher, Proceedings of the 2010 Workshop on Procedural Content Generation in Games, 2010, <https://dl.acm.org/doi/epdf/10.1145/1814256.1814266> [Accessed 4/20/2026]
- [5] P. Fraile-Jurado, “Geographical Aspects of Open-World Video Games”, ResearchGate, Games and Culture, 2023, University of Seville, https://www.researchgate.net/publication/384358902_Geographical_Aspects_of_Open-World_Video_Games [Accessed 5/8/2026]
- [6] J. Freiknecht, “Procedural content generation for games”, No Known Publisher, Universität Mannheim, 2021, <https://madoc.bib.uni-mannheim.de/59000/1/Procedural%20Content%20Generation%20for%20Games.pdf> [Accessed 5/9/2026]
- [7] M. Hendrikx, S. Meijer, J. Van Der Velden, and A. Iosup, “Procedural Content Generation for Games: A Survey”, ACM, ACM Transactions on Multimedia Computing, Communications, and Applications, 2013, Delft University of Technology, <https://dl.acm.org/doi/epdf/10.1145/2422956.2422957> [Accessed 5/9/2026]
- [8] S. Barrett, “stb”, GitHub: nothings/stb, 2010-2015. https://github.com/nothings/stb/blob/master/stb_image_write.h [Accessed 4/25/2026]
- [9] “Dig-Dug”, Wikipedia, Last edited: 5/8/2026 https://en.wikipedia.org/wiki/Dig_Dug [Accessed 5/8/2026]
- [10] Avery (Unknown Full Name), YouTube, 2022, <https://www.youtube.com/watch?v=S9QTsf9AUqQ> [Accessed 4/20/2026]
- [11] E. F. Codd, “Cellular Automata”, IBM Corporation, Academic Press Inc, 1968, Florida, United States of America, <https://dl.acm.org/doi/book/10.5555/1098682> [Accessed 5/8/2026]